Scalable Dataset Splitting through Constrained Low-Rank Similarity Optimization

Ruijie Zhu

Abstract

Similarity-aware dataset splitting reduces overlap between training and evaluation data while controlling fold sizes. Optimizing similarity across all entity pairs usually requires a matrix with quadratic storage. We present a framework that evaluates and optimizes this dense objective through compact low-rank factors, with explicit size and group constraints. Across six molecular datasets, the largest observed rank-256 approximation error is 2.71% of the true cut for ordinary Tanimoto similarity. Powered kernels have substantially larger errors, so their accuracy must be checked separately. Direct dense-kernel controls use identical initial assignments and optimization procedures, separating representation error from the quality of the returned partitions. On 32,000 ChEMBL fingerprints, rank 256 takes 8.7 seconds versus 82.7 seconds for the dense arm on two CPU cores, with similar original-kernel objectives. A separate 32-core protocol partitions two million fingerprints in a median 4.1 minutes at a 1% size tolerance. Comparisons with DataSAIL and CCPart show that grouping and capacity constraints materially affect conclusions about quality. Reducing aggregate overlap does not consistently reduce nearest-training exposure; matching DataSAIL's groups removes the average global advantage observed when only fold counts match. A metadata-only ODAC25 example illustrates composition holdouts through group representatives. These results establish a scalable constrained optimization framework with measured accuracy–cost trade-offs, rather than a universal ordering of splitting methods or a guarantee of real-world predictive performance.

Keywords: dataset splitting; similarity; Nyström approximation; model evaluation; molecular machine learning.

1 Introduction

A dataset split defines the generalization task used to evaluate a model. Predicting another member of a known chemical series differs from predicting members of a new series. Random, scaffold, cluster and temporal splits impose different boundaries between training and evaluation data and can change reported model performance [Wu et al., 2018, Yang et al., 2019]. Lo-Hi distinguishes lead optimization from novel-hit identification [Steshin, 2023], while temporal studies and SIMPD relate evaluation design to the project scenario [Sheridan, 2013, Landrum et al., 2023]. A useful split must therefore be both computationally feasible and appropriate for the intended transfer task.

One way to construct a split is to minimize similarity between folds subject to constraints on their sizes. A dense objective includes every entity pair, but storing its matrix requires quadratic memory. DataSAIL reduces this problem through clustering and integer programming [Joeres et al., 2025, 2026]. A nearest-neighbour graph instead retains selected pairwise relationships. We examine a complementary approach: compress the dense similarity matrix so that its cross-fold objective can be evaluated and optimized without explicit all-pairs storage.

We assign each entity a short vector whose dot product approximates its similarity to another entity. The cross-fold objective can then be calculated from the sum of vectors in each fold (Figure 1). Nyström approximation and its connections to kernel clustering are established [Williams and Seeger, 2000, Dhillon et al., 2004, Gittens and Mahoney, 2016, Wang et al., 2019, Chitta et al., 2011]. Our contribution is a framework that combines this representation with fold-size constraints, practical optimization, and diagnostics for approximation accuracy and constraint feasibility. Its computational benefit depends on whether the chosen kernel can be represented accurately at an affordable rank.

The algorithmic question is therefore whether compact factors support useful constrained optimization at an affordable computational cost. We address it with direct dense-kernel controls using the same search routines, component ablations of the current implementation, and independently evaluated quality–cost comparisons. CCPart supplies a complementary threshold-based connected-component contract [Fernández Díaz et al., 2024]. Its native groups and realized sizes are evaluated explicitly rather than treated as interchangeable with entity-level constraints.

Total overlap and close-analogue exposure are different objectives. A few highly similar pairs may contribute little to a sum over all pairs while strongly affecting prediction. We therefore measure both global overlap and each test entity's similarity to its closest training neighbour. Kernel powers emphasize strong similarities, and capped groups encourage nearby entities to remain together. These extensions change the separation achieved, the approximation required and the computational cost.

We evaluate the framework through four connected questions. First, does the factor preserve the dense objective? Second, what separation does optimization achieve under matched constraints? Third, how do fold-size tolerance and dataset size affect cost? Finally, when does the resulting split reflect the intended transfer task? Six molecular datasets provide the primary comparison, ChEMBL provides scaling measurements, and independent synthetic cohorts test the relation between split design and model selection. An ODAC25 application examines metadata-only composition holdouts at the original configuration counts (Section 3.6).

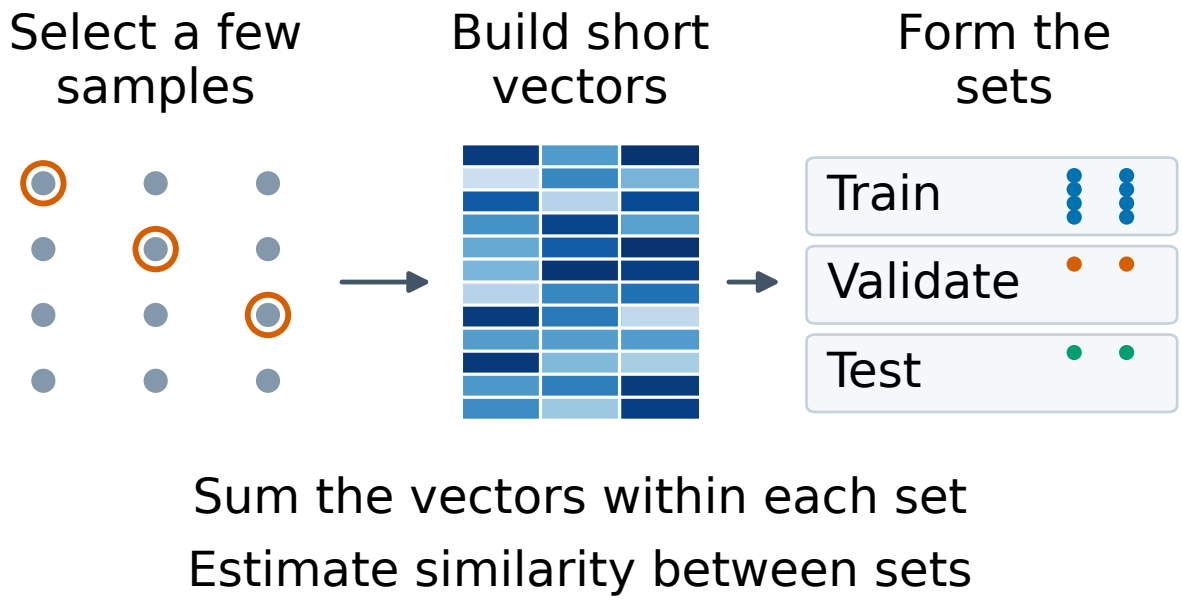


Figure 1: How the split is constructed. A small set of reference samples is selected (orange circles), and each sample is represented by a short vector. Samples are assigned to sets of the requested sizes. Summing the vectors within each set allows similarity between sets to be estimated without storing every pairwise comparison (Equation 3). The dots and matrix entries illustrate the workflow; they are not experimental results.

2 Low-rank similarity splitting

2.1 Objective and separation measures

Let $S \in \mathbb{R}^{n \times n}$ be a symmetric, nonnegative, positive-semidefinite similarity kernel, and let z_i denote entity i's fold. Define the unordered cross-fold cut and its normalized overlap as

$$\mathcal{C}_S(z) = \sum_{i<j} S_{ij} \mathbf{1}[z_i \neq z_j], \qquad L_S(z) = \frac{\mathcal{C}_S(z)}{\sum_{i<j} S_{ij}}. \tag{1}$$

The denominator is independent of the partition, and diagonal entries do not contribute. We report the ratio only when the denominator is positive. Nonnegative similarity is essential to this interpretation. Positive semidefiniteness alone does not make a signed cosine kernel an overlap measure.

For actual fold counts n_c, the random expectation of the global fraction is

$$\mathbb{E}[L_S] = 1 - \frac{\sum_c n_c(n_c - 1)}{n(n-1)}. \tag{2}$$

The primary global score is $L_S / \mathbb{E}[L_S]$. The primary local score is the raw mean NNL $= |T|^{-1} \sum_{i \in T} \max_{j \in R} S_{ij}$, where T is the test fold and R the training fold. Lower values indicate less overlap.

2.2 Low-rank representation

We construct a Nyström factor from r landmarks. If C contains entity–landmark similarities and $W = U\Lambda U^\top$ is the landmark kernel, retaining its numerically positive eigenvalues gives $B = CU_+\Lambda_+^{-1/2}$ and $\widetilde{S} = BB^\top$. For fold sums $p_c = \sum_{i:z_i=c} B_i$, the cut of the approximate kernel is given exactly by

$$\mathcal{C}_{\widetilde{S}}(z) = \frac{1}{2}\left(\left\|\sum_c p_c\right\|^2 - \sum_c \|p_c\|^2\right). \tag{3}$$

Factor storage and fold-sum construction require $O(nr)$ space and work, respectively. This identity makes repeated objective evaluation practical without constructing $\widetilde{S}$.

When fold sizes are equal and fixed, minimizing this cut has the same optima as kernel k-means. Unequal capacities weight within-fold similarity differently, so standard kernel k-means guarantees do not transfer directly. The supplement derives the objective identity and a conditional approximation-error bound.

2.3 Constrained optimization and accuracy checks

Largest-remainder rounding converts requested proportions into integer targets summing to n. Zero tolerance enforces those targets exactly; positive tolerance permits a declared two-sided corridor relative to each requested fold size, with integer rounding. Each seeded restart alternates between calculating fold sums and proposing assignments that satisfy the size constraints. Proposals are accepted only if they do not increase the factor objective. A limited set of candidates is then refined before the final partition is selected. Legal single moves operate within a corridor; swaps preserve exact counts [Fiduccia and Mattheyses, 1982, Kernighan and Lin, 1970]. The standard setting uses four restarts, at most 25 assignment iterations and two refined candidates. The optimizer remains heuristic.

Rank can be fixed or selected using a bounded accuracy budget. The optional joint policy increases the number of sampled pairs when uncertainty prevents a decision. It increases rank when the estimated error is established as too large or the pair-sampling budget is exhausted. It accounts for all checks over a declared rank–sample grid and uses a separate final audit. The cut criterion measures error for the returned partition, normalized by the kernel's total absolute off-diagonal mass. Establishing this tolerance does not certify a uniform approximation over all partitions or a near-optimal partition. Budget failure is reported explicitly.

2.4 Controlling strong local relationships

For integer $p \geq 1$, we can factorize the elementwise power $S^{\circ p}$. The Schur product theorem preserves positive semidefiniteness. Powering reduces weak similarities relative to strong ones: $0.1^3 = 0.001$, whereas $0.7^3 = 0.343$. If $N_\tau(z)$ counts crossing pairs with similarity at least $\tau > 0$, then $\mathcal{C}_{S^{\circ p}}(z) \geq \tau^p N_\tau(z)$. The powered cut consequently provides a soft penalty on strong crossing pairs. It is distinct from the mean nearest-neighbour evaluation score, and its approximation requirements must be checked separately.

Grouping restricts which assignments are permitted. We process top-32-neighbour edges above a threshold in decreasing similarity, merging components subject to a size cap. A group's factor is the sum of its members' rows; its weight is their count. Group contraction therefore preserves the factor objective exactly. The combined molecular setting uses $p = 3$, threshold 0.4, and a cap of half the smallest target fold, with component growth frozen after a cap-blocked merge. We refer to the unpowered, ungrouped method as the plain setting and to the powered, grouped method as the combined setting.

The size-constraint policy determines what happens when whole groups cannot be assigned within the fold capacities. The default count-first policy may dissolve groups and reports that relaxation. The strict policy preserves every group or reports infeasibility or an unresolved feasibility check. Neither capped grouping nor a powered objective guarantees that every pair above the threshold stays within one fold. All reported separation scores use the original, unpowered kernel.

2.5 Computational scope

The factor avoids storing a quadratic-size matrix, but some computations can still require quadratic work. Input features still require $O(nd)$ space and can dominate process memory. Factor construction, assignment sorting and the number of accepted refinement moves determine runtime. Exact neighbour construction adds quadratic pairwise arithmetic even when temporary memory is blocked. We therefore report scaling separately for the plain and combined settings. Refinement under exact counts can also cost substantially more than refinement within a relaxed corridor. Each timing and memory measurement states which operations it includes.

3 Results

3.1 Accuracy of the compressed objective

We first assess how accurately the factor represents the intended cut objective. Across six full molecular datasets, three seeds and five partitions per factor, the largest observed rank-256 error is 0.0070 of the unpowered kernel's off-diagonal mass. The corresponding error for the cubed kernel is 0.1367. At rank 512 these values are 0.0027 and 0.0936, respectively (Figure 2). Accuracy for the unpowered kernel therefore cannot be assumed for the powered objective. A coarse powered factor can still produce useful local separation, but that behavior must be measured on the returned partition rather than inferred from unpowered-kernel accuracy.

Exhaustive small-instance comparisons separate approximation error from optimization error. On twenty 12-entity instances, we enumerate 2,970 admissible ungrouped assignments and thirty assignments with paired entities as indivisible groups. Even with all twelve landmarks, the heuristic has nonzero optimization gaps on some ungrouped instances: the largest gaps are 0.0132 and 0.0408 of kernel mass for the unpowered and cubed kernels. The grouped full-landmark cases reach their enumerated optima in this small suite. Increasing rank can therefore remove approximation error while leaving a separate optimization error.

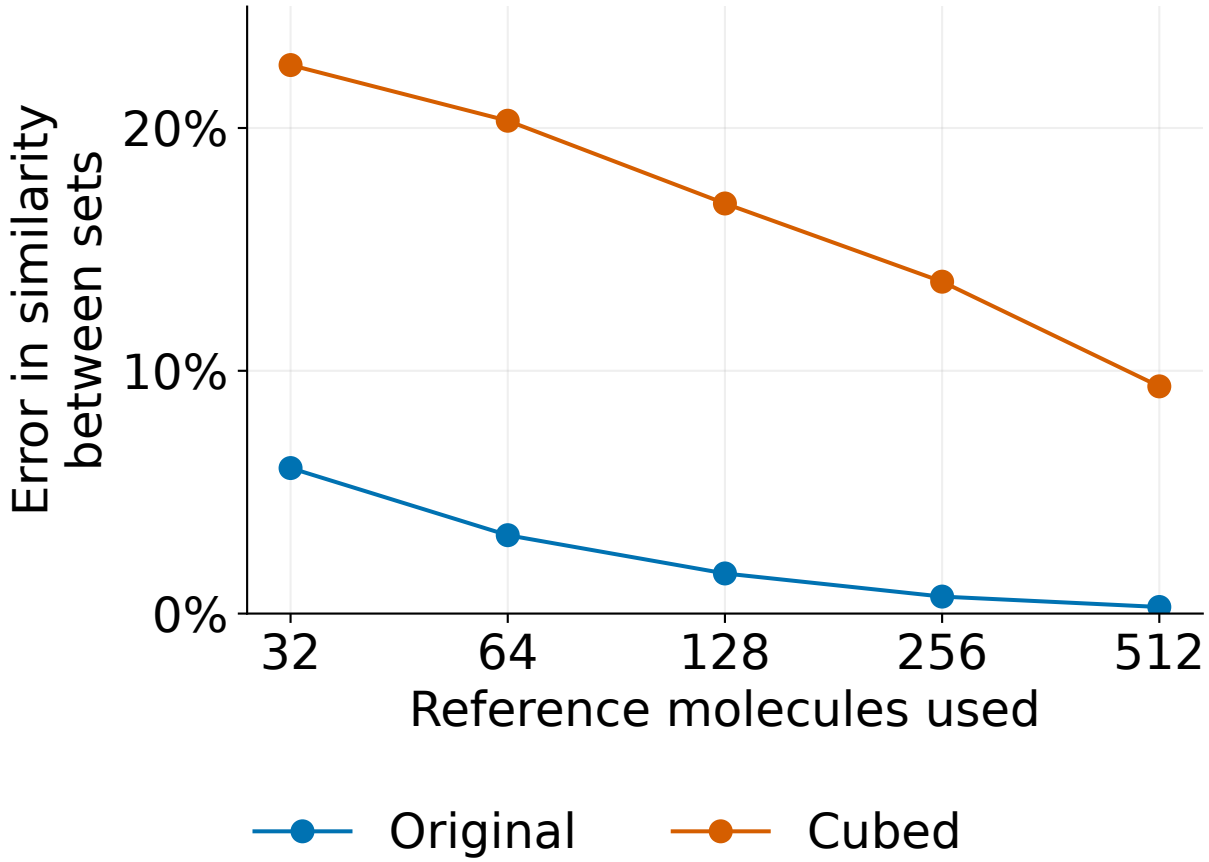


Figure 2: Accuracy as the number of reference molecules increases. Error is the absolute difference between the approximate and full similarity summed across sets, expressed as a percentage of the total similarity over all distinct pairs. Each point is the largest error across six datasets, three random seeds and five partitions: the optimized partition and four changes that preserve set sizes. The blue line uses the original similarities; the orange line cubes them before approximation. The percentage for each line uses the total similarity for its own definition. These are errors observed in the tested partitions, not guarantees for every possible partition.

The complete decomposition and accuracy–cost controls appear in the supplement.

3.2 Direct dense-kernel optimization controls

Compression fidelity and optimization quality answer different questions. The six-dataset controls compare the factorized and direct dense implementations with identical starting assignments, capacity constraints, search routines and iteration limits. All final quality values are evaluated with the original kernel. At rank 256 and a 1% size corridor, the mean ratio of the factor solution's true cut to the dense solution's true cut is 0.9996, averaging seeds within each dataset before averaging the six datasets; per-dataset quality and timing values are in the supplement. Neither heuristic solution establishes an optimum. Higher rank can improve objective fidelity without producing a monotonically better optimized partition.

The comparison also exposes a size-dependent computational benefit. Dense optimization is faster on the smaller molecular datasets, while compact factors become faster as the number of entities grows. At 32,000 ChEMBL fingerprints, rank 256 takes 8.7 seconds versus 82.7 seconds for the dense arm, with mean peak process memory of 1.8 versus 12.0 GiB (Table 1). These two-core controls use the same initial assignments and search limits. They are separate from the four-restart, 32-core two-million-entity public-API experiment below.

Entities	Dense (s)	Factor (s)	Dense GiB	Factor GiB	Cut ratio
4,000	1.38	1.08	0.92	0.82	1.0003
16,000	23.79	4.52	3.93	1.36	1.0004
32,000	82.68	8.74	12.04	1.83	1.0004

Table 1: Dense versus rank-256 factor optimization on nested ChEMBL samples, using identical initial assignments and search limits on two CPU cores with a 1% size corridor. Time includes representation construction and optimization; peak process RSS precedes independent scoring. Cut ratio divides the factor solution's original-kernel cut by the dense solution's cut. Entries average three seeds. Both solutions are heuristic.

Equal-time controls complement the iteration-matched results. Their timed optimizer is stopped externally at the deadline, and only a feasible checkpoint completed before that deadline is scored. Representation construction consumes the same budget as search. The supplement also retains the earlier cooperative-deadline sensitivity cases and their overruns.

Error relative to the optimized cut can be appreciably larger than error relative to all kernel mass. Reanalysing the 900 existing full-kernel comparisons, the largest rank-256 cut-relative errors are 2.71% for ordinary similarity and 64.47% for cubed similarity. These are approximation errors at tested partitions, not optimization gaps or uniform guarantees. The supplementary tables retain both normalizations and distinguish the returned partition from count-preserving perturbations.

Current-version component controls further separate feasible initialization, balanced assignment, local refinement, restart budget and landmark sampling. Assignment accounts for most of the observed global reduction: the average score changes from 1.001 for feasible initialization to 0.626 for assignment alone, versus 0.625 with four restarts and two refined

candidates. Extra search does not consistently improve nearest-training exposure, because that is not the ordinary factor objective. The grouping controls fix and strictly preserve the same groups before comparing their feasible initialization with ordinary- and cubed-factor optimization. Their complete global, local and timing results accompany the rank and dense controls in the supplement.

3.3 Separation under matched constraints

Optimization improves separation beyond that obtained by grouping alone. With the entities, groups and exact fold counts fixed, factor-space optimization improves both global overlap and mean nearest-training similarity on all six datasets, relative to a seeded feasible assignment (Table 2). Every group is preserved in these controls. This control measures the improvement from optimization after groups have been fixed. It does not attribute every advantage of the combined pipeline to compression.

Assignment	Global overlap / random	Mean nearest similarity
Feasible groups	0.959	0.387
Groups + ordinary factor	0.791	0.306
Groups + cubed factor	0.745	0.292

Table 2: Optimization beyond grouping. Every comparison retains the same entities, constructed groups and exact 80/10/10 counts. Values average six dataset means after averaging three seeds. The factor optimizer starts from the feasible control and preserves every group. Both optimized settings improve both measures on all six datasets. Lower values indicate less overlap.

The unpowered factor directly targets similarity summed across all entity pairs. Its mean global score is 0.625, compared with 0.791 for the top-32 graph, after averaging seeds within each dataset. The combined setting reduces nearest-training exposure relative to the plain setting, while the top-32 graph achieves lower mean exposure than the combined setting (0.254 versus 0.282). Changing graph degree also changes the available trade-off (Figure 3). Thus, the settings occupy different positions on the global–local separation trade-off rather than giving a consistent ordering across both criteria.

DataSAIL provides a closer comparison for the dense objective and shows why constraints on admissible partitions matter. Sixteen of its twenty-four prespecified configurations return assignments; eight report infeasibility under their cluster and capacity settings. At exactly matched native counts, entity-level low rank has mean global score 0.577, versus 0.666 for DataSAIL. Requiring low rank to preserve the groups passed to DataSAIL's solver changes its mean to 0.668, eliminating that average quality advantage. These summaries weight the four datasets equally after averaging their configurations.

The apparent advantage at matched counts therefore depends on the admissible partitions. Even after groups are matched, DataSAIL's linked cluster similarities differ from sums of entity-level similarities, and the optimizers differ. The methods are consequently not solving an identical optimization problem. It shows how indivisibility constraints affect

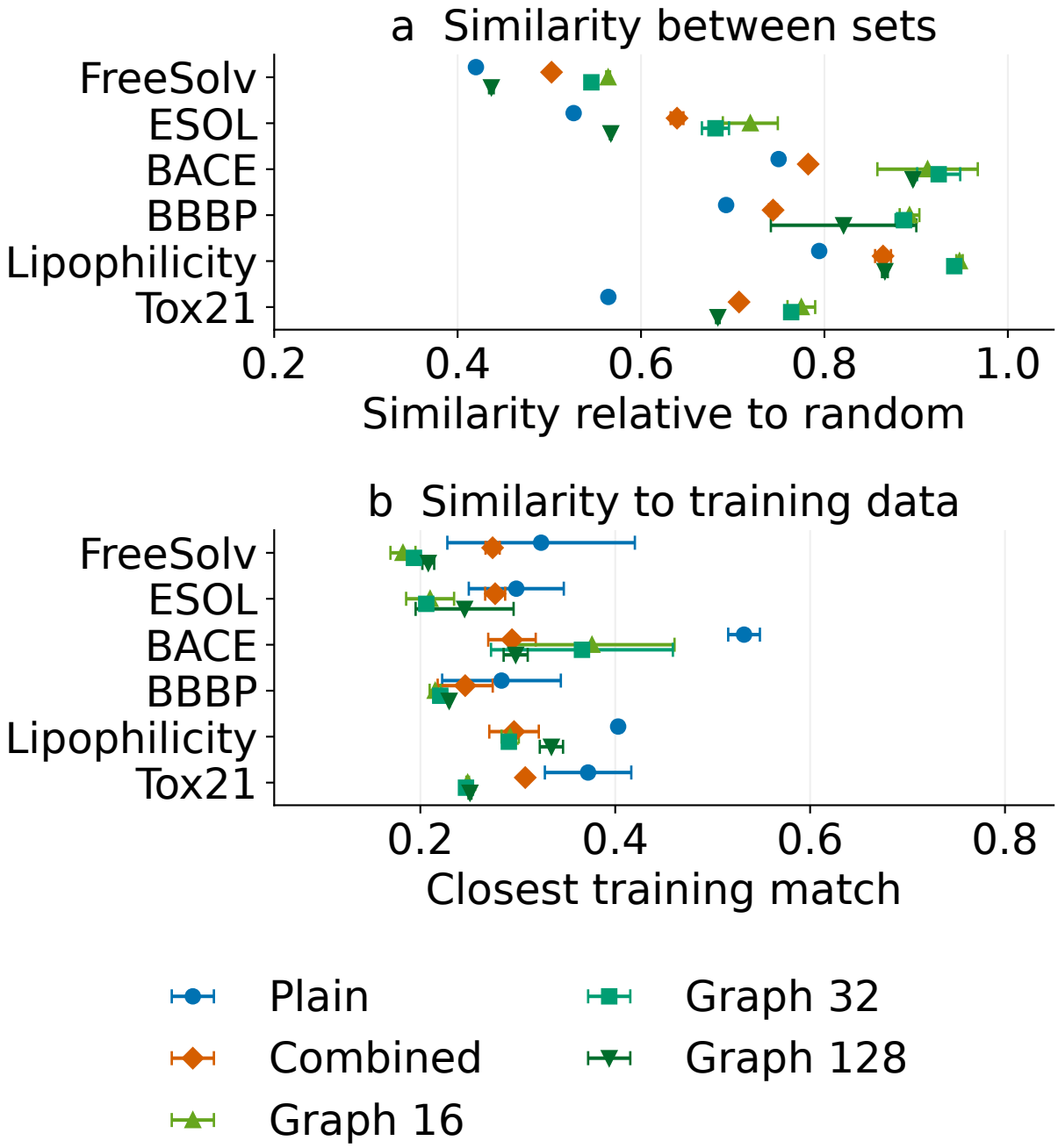


Figure 3: Two measures of separation on six molecular datasets. (a) Similarity summed between sets, divided by its expected value for a random assignment with the same set sizes. (b) Mean similarity of each test molecule to its closest training molecule. Lower values indicate less similarity. Points show means of three random seeds; bars show their standard deviations. All methods retain the same molecules and use a 1% size tolerance. Plain and combined settings are defined in the text. Graph labels give the number of closest neighbours retained per molecule. Graph construction and size repair are included in the reported runtime. Both panels use original Tanimoto similarities.

the separation achieved by entity-level dense-overlap optimization. The supplement reports native outcomes, timings, group-preserving controls and memory profiles.

CCPart supplies a different grouping-based comparison (Table 3). Its connected components preserve threshold relationships, whereas its native packing realizes its own fold sizes. The two low-rank interventions start from those assignments and either retain only the counts or also preserve every component. These comparisons show the effect of optimizing aggregate overlap within, or after relaxing, a declared component constraint. Threshold choice, realized sizes and similarity construction cost remain visible. The results do not establish a universal ordering of the methods on global overlap and nearest-training exposure.

At threshold 0.4, optimizing while preserving components changes mean global overlap from 0.835 to 0.784, with nearest-training similarity nearly unchanged at 0.292 versus 0.291. Retaining only the native counts instead gives global overlap 0.622 but nearest-training similarity 0.364. This example makes the constraint distinction concrete: allowing more partitions can improve the aggregate objective while increasing close-neighbour exposure.

Threshold	Assignment	Global	Nearest	Seconds
0.4	CCPart	0.835	0.292	0.65
0.4	Low rank, counts	0.622	0.364	0.86
0.4	Low rank, groups	0.784	0.291	0.62
0.6	CCPart	0.924	0.414	0.62
0.6	Low rank, counts	0.625	0.371	0.80
0.6	Low rank, groups	0.682	0.341	0.59
0.8	CCPart	0.979	0.560	0.66
0.8	Low rank, counts	0.626	0.364	0.82
0.8	Low rank, groups	0.646	0.402	0.78

Table 3: CCPart and low-rank controls at its native counts, with and without its connected-component groups. Global is overlap relative to count-matched random; Nearest is mean maximum test-to-training Tanimoto similarity. Scores average input-order repetitions within datasets and datasets equally. Native runtime includes exhaustive threshold-similarity construction and component packing. Matched controls include factor construction and optimization; native groups are reused, so their clocks describe an optimization intervention, not a standalone grouping pipeline. Realized counts, retention, missing folds and per-dataset results are in the supplement.

3.4 Runtime, dataset size and fold-size tolerance

At 100,000 ChEMBL fingerprints on two CPU cores, the median plain split-call time is 567.2 seconds with exact counts and 40.1 seconds with a 1% corridor. The inputs, rank and restart settings are otherwise matched. The difference reflects the cost of feasible refinement, including the use of swaps at exact capacities (Figure 4). Linear factor storage therefore does not imply the same runtime at different capacity tolerances.

The plain implementation partitions two million fingerprints in a median 243.9 seconds using 32 CPU cores, with peak process RSS of 58.8 GiB. All three cases retain all entities

and satisfy the declared 1% corridor. This is a separate hardware configuration from the paired two-core comparison; the difference is not interpreted as an algorithmic speedup.

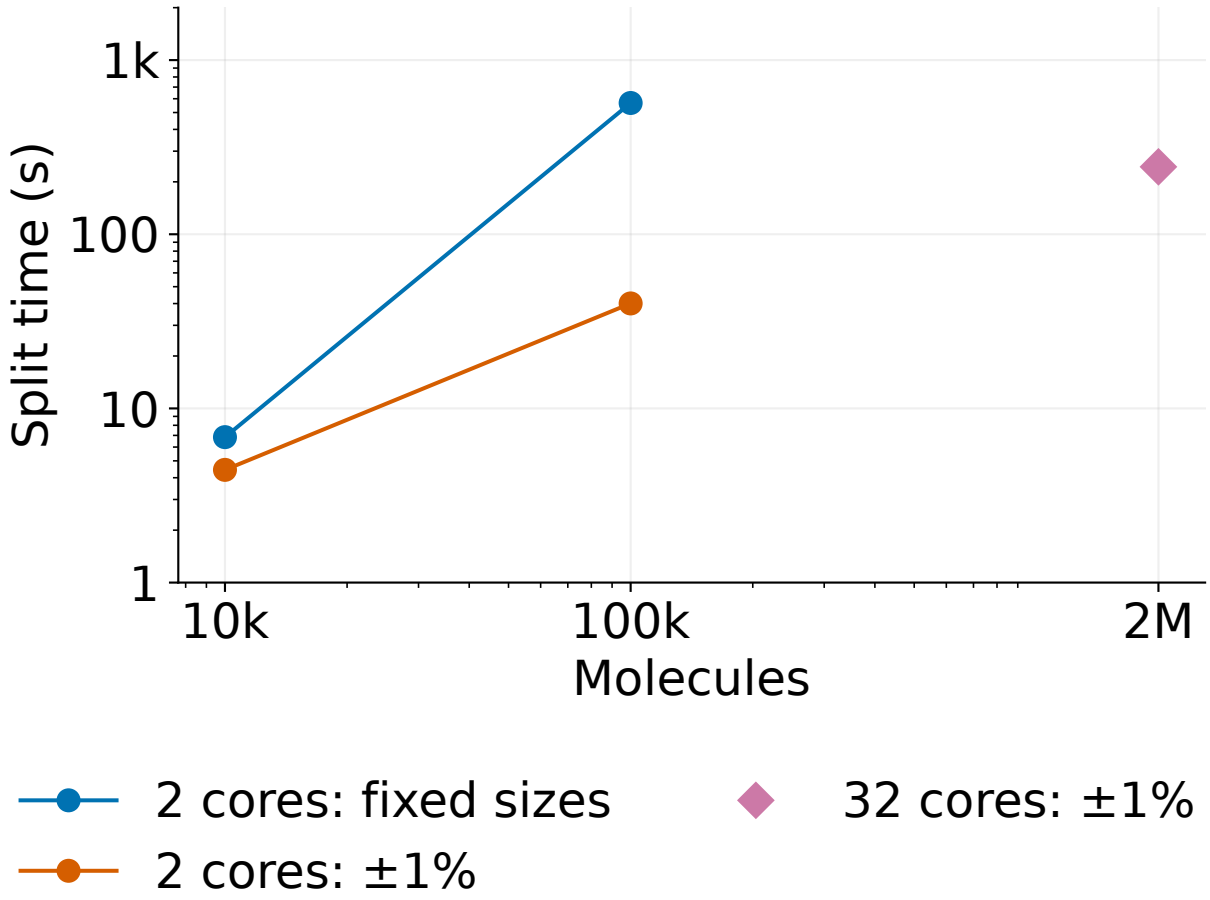


Figure 4: Plain split-call time on nested ChEMBL samples, using rank 256, four restarts and two refined candidates. Two-core cases run on lambda2; the separate 32-core point runs on lambda13. Points are medians of completed cases. Warm-up, feature loading and independent auditing are outside the split clock. The full case inventory, including end-to-end budget failures and the distinct combined-setting range, appears in the supplement. An end-to-end failure is not assigned a split-call runtime.

The large-data protocol evaluates separation and approximation accuracy as well as computational cost. For every completed 100,000- and two-million-entity plain case, the independently sampled returned-cut error interval has an upper endpoint below 0.008 of unpowered kernel mass. Each uniformly sampled test query is compared with every training entity. At two million entities, mean nearest-training similarity is 0.618 across the three sets of sampled queries, showing that low global overlap does not remove close-analogue exposure. Sample sizes and conservative query-sampling intervals accompany the results.

3.5 Matching the split to the transfer task

Independent cohorts allow us to test which transfer task a split reflects. In the synthetic family model, random splitting identifies the better model for known-family interpolation in all twenty generated instances, but identifies the better model for new-family transfer in none. The combined split shows the reverse pattern: agreement in all twenty new-family instances and none of the interpolation instances. Plain splitting lies between these outcomes (Figure 5).

The direction of evaluation error also matters. Random evaluation is optimistic about the family-sensitive model on the new-family cohort, whereas the combined split is pessimistic about that model's interpolation performance. Stronger separation is useful when it matches the intended transfer boundary. Its lower held-out score alone does not make it a more realistic test.

An earlier-release assay study illustrates a related risk in real molecular data. Across ten prespecified external assays, the combined setting gives mean random-forest ROC-AUC 0.522. A label-balance sensitivity intervention at unchanged counts raises this to 0.757. The intervention also changes structural composition and may break groups. It therefore cannot isolate the causal contribution of label proportions. These archival results demonstrate why target composition and absolute predictive scores should accompany similarity measures. They supply no observed future outcomes against which to validate a deployment claim.

3.6 A metadata-only composition holdout for ODAC25

We next apply the framework to a composition holdout in the atomistic ODAC25 release [Sriram et al., 2026]. The aim is to separate compositions at the original training and validation counts using only metadata. Reading names, formulas and multiplicities from pinned ColabFit Parquet snapshots requires a transfer of 63.4 MB of production payload. After restoring multiplicities, the 37,456,852 retained rows represent 37,711,674 configurations. These totals exactly match the published filtered training and validation counts. Small pilot and documentation requests are additional; coordinates, forces and energy labels are not read. Conservative rules based on family names, shared configuration aliases and identical compositions define 2,050 indivisible components. A versioned weighted-factor adapter then constructs a two-fold holdout at exactly 36,920,036 training and 791,638 validation configurations.

On the recorded elemental-fraction vectors, the count-normalized global overlap falls from 0.969 for the original split to 0.283 after low-rank optimization. Mean nearest-training similarity falls from 0.846 to 0.194–0.196 (Table 4). The same-group feasible initialization already reduces overlap, but further optimization adds substantial separation. A dense-kernel control is competitive, so this small representative problem does not establish a low-rank speed advantage. Thus, the millions of configurations describe how many records receive an assignment; optimization operates on 2,050 representatives.

The resulting boundary is a strong composition-shift test. Phosphorus occurs in 36.8% of validation configurations versus 1.6% of training configurations for seed zero, and two components contribute 40.6% of validation weight. Zero identical- composition overlap follows from the imposed grouping contract. These results demonstrate that the specified holdout

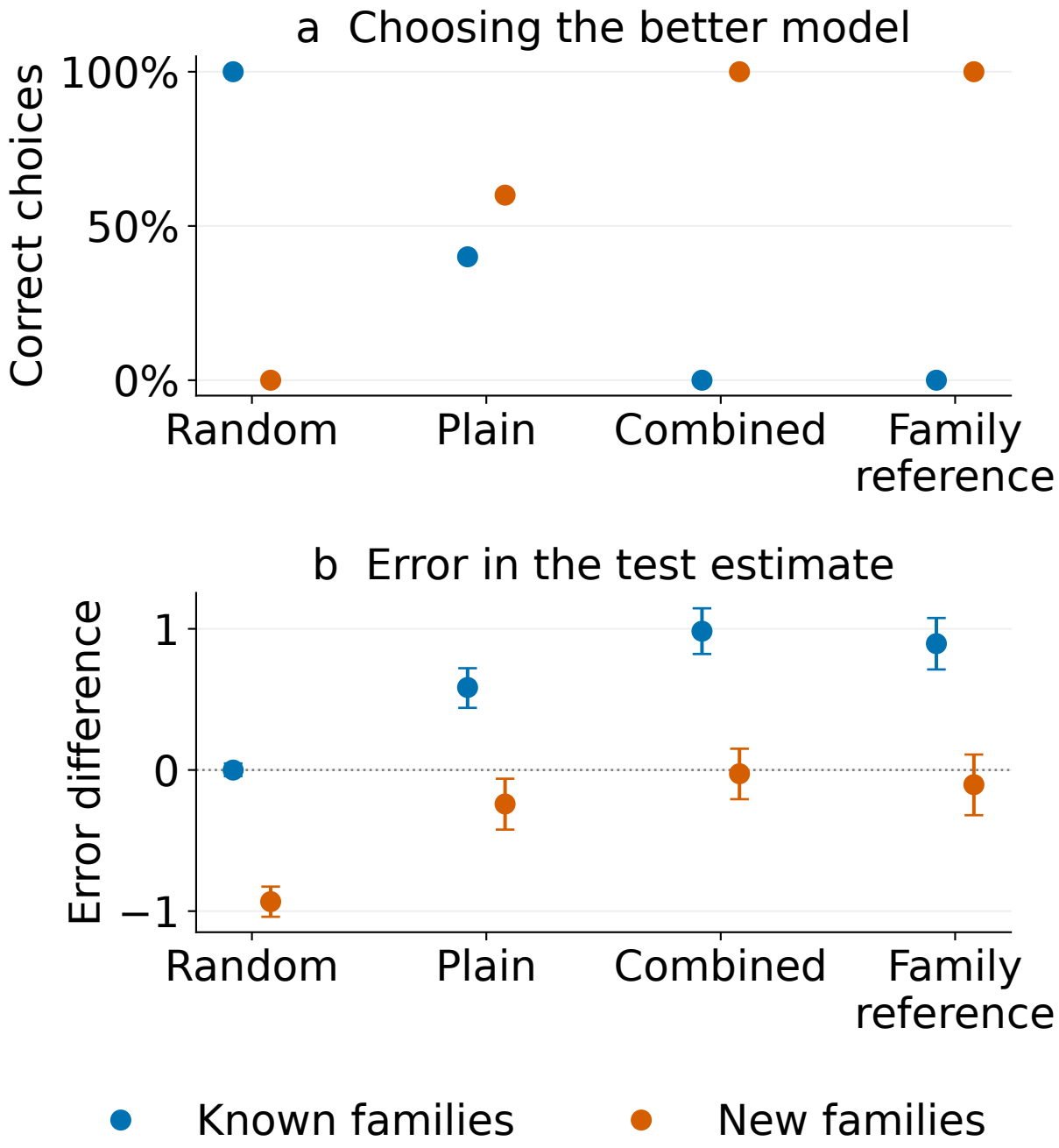


Figure 5: Does the split identify the better model for the intended task? (a) Percentage of twenty synthetic instances in which the test set and an independent cohort give the same model ranking. (b) Test-set root mean square error minus error on the independent cohort for the family-sensitive model; negative values underestimate the independent-cohort error. In (b), bars show 95% bootstrap intervals across instances. Known-family cohorts contain families present in training; new-family cohorts contain unseen families. The family reference uses the generator's family labels. The ridge model is given the stable generative features. This synthetic study does not establish performance on future real projects.

Assignment	Global ratio ↓	Mean nearest ↓
Original	0.969	0.846
Groups before optimization	0.825–0.873	0.526–0.574
Low-rank optimization	0.283	0.194–0.196
Dense-kernel control	0.287	0.154–0.164

Table 4: ODAC25 composition separation at identical configuration counts. The global ratio is normalized by the random expectation at those counts. Scores use recorded elemental fractions, configuration weights, 200,000 pair draws and 512 validation queries against every distinct training formula. Ranges span three optimizer seeds, which yield only two distinct low-rank assignments. The group control uses the same exact-count feasible initialization; it is not a uniform sampler of feasible partitions.

can be constructed using a small metadata transfer. Its relevance to prediction and deployment remains untested. The ODAC25 section of the Supplementary Information gives the weighting, aggregation, source versions, individual seeds and numerical checks.

4 Discussion

The framework makes a dense similarity objective practical to evaluate and optimize under explicit fold-size constraints. The results separate three questions that are often conflated: how accurately the factor represents the kernel, how effectively the optimizer partitions it, and whether the resulting split addresses the intended prediction task. The unpowered kernel is accurately represented at modest rank in the examined datasets, but exhaustive small instances show that optimizer suboptimality can remain. Matched-group controls show that optimization improves separation after groups are fixed. The DataSAIL comparison also shows that matching fold counts alone is insufficient to attribute a quality advantage to the representation or optimizer.

The direct dense comparison holds initialization, capacity rules and search procedures fixed. It distinguishes the true-kernel quality of the returned partitions from the fidelity of their approximate objective values. In small datasets, factor construction can cost more than storing and optimizing the dense kernel. The purpose of compression is to extend the affordable problem size; neither a low rank nor linear factor storage guarantees a speed advantage at every dataset size. Native process memory and representation-array storage are therefore reported separately.

The appropriate separation criterion depends on the transfer task. Plain low rank reduces similarity summed across all pairs. Powers and groups strengthen close-analogue separation while changing global overlap, accuracy requirements and cost; a sparse graph can provide stronger local separation for a given configuration. These are observed differences among tested settings, not a theorem that improving global separation must worsen local separation. The synthetic study illustrates the consequence for evaluation: a split suited to new-family transfer can be pessimistic for known-family interpolation. Lower test scores alone therefore do not establish a more useful benchmark. A practical analysis should specify the intended

transfer task, representation, fold sizes and retention policy before examining final model outcomes, and report global overlap, nearest-training similarity and target composition together. Label-informed changes require a declared benchmark-design or sensitivity-analysis role and a separate final evaluation.

The ODAC25 case demonstrates how metadata can support a specified composition holdout without reading atomistic coordinates or response labels. Its assignment coverage and its optimization size describe different quantities: the method assigns millions of configurations through a small number of group representatives. The chemical shift, concentration of validation weight and competitive dense control limit what can be inferred from this application. The separate ChEMBL experiments provide the computational scaling evidence.

Several limitations guide further work. The datasets are convenience samples, and molecular separation depends on the fingerprint representation. Powered kernels can require substantially more rank, strict whole-group capacities can be infeasible, and linear factor storage does not imply fast construction at every size or tolerance. Accuracy auditing also has a computational cost. On the three development datasets used for equal-confidence probe comparisons, a fixed 32,768-pair budget was faster than joint adaptive allocation and established the same requested tolerance in every seed. These results support fixed probes as the default; the optional adaptive policy has no established efficiency advantage in this comparison. Finally, the synthetic invariant model receives the generative stable features. Real deployment validation requires independently specified project tasks and genuine dated outcomes, which the synthetic and simulated-temporal studies do not provide.

5 Methods

5.1 Datasets and representations

The six full molecular datasets are FreeSolv, ESOL, BACE, BBBP, Lipophilicity and Tox21 [Wu et al., 2018]. Radius-two Morgan fingerprints have 1024 bits, with binary Tanimoto similarity [Rogers and Hahn, 2010, Ralaivola et al., 2005]. Structural optimization does not use response values and retains every input entity for which features were successfully generated. Raw-input, feature, source and assignment hashes identify the exact observations. The supplementary earlier-release studies include the eleven-dataset molecular comparison, two composition-descriptor materials checks, neural-model reuse and external assays.

5.2 Controls and established comparators

The group mechanism study uses three seeds and exact 80/10/10 counts. For each dataset and seed, a single set of capped groups is fixed. The control is a seeded assignment that satisfies the fold capacities and is constructed without using factor values. Optimization with unpowered and cubed factors starts from this assignment and preserves every group. Exact small-instance references use twenty 12-entity binary datasets, counts 8/2/2, powers one and three, and landmark counts 2, 4, 8 and 12. Enumerating all feasible assignments measures approximation and optimization gaps separately, both without groups and with pairs as indivisible groups.

DataSAIL is the established comparator for reducing dense cross-fold similarity [Joeres et al., 2025, 2026]. An ordinary weighted k-nearest-neighbour graph tests the alternative of selecting pairs rather than compressing their similarities. We use Mt-KaHyPar's Graph API, QUALITY preset and CUT objective [Gottesbüren et al., 2024], with degrees 16, 32 and 128. Exact top-k construction, safe integer weight encoding, partitioning and any repairs needed to satisfy lower and upper count bounds are included in the split time. Native and repaired assignments are retained separately.

The six-dataset graph and low-rank comparison uses three split seeds, 8:1:1 proportions and tolerance 0.01. DataSAIL 1.3.0 uses C1e with SCIP on FreeSolv, ESOL, BACE and BBBP. Its prespecified grid combines cluster counts 9, $\max(9, \lfloor n/50 \rfloor)$ and $\max(9, \lfloor n/20 \rfloor)$ with solver budgets of 60 and 300 seconds, $\delta = 0.1$ and $\epsilon = 0.01$. For every native assignment, we construct an entity-level low-rank control with identical counts and a second control that also preserves the groups passed to the native solver. Equal groups do not make DataSAIL's linked cluster similarities identical to the entity-level dense objective. We report solver-declared infeasibility as an outcome and do not impute a score.

Lo-Hi, UMAP/Ward, scaffold and Butina comparisons address additional retention or grouping contracts and remain in the explicitly versioned archival studies [Steshin, 2023, Hernández-Hernández and Ballester, 2023, Guo et al., 2025, Bemis and Murcko, 1996, Butina, 1999]. Their retained subsets and realized counts accompany the comparisons. We report these comparisons separately from controls with equal constraints.

5.3 Fidelity, accuracy budgets and computation

We assess full-kernel fidelity on six datasets using three factor seeds, five landmark counts and powers one and three. For each setting, we evaluate the current returned partition and four count-preserving perturbations against an independent full NumPy Tanimoto kernel. Absolute cut error is divided by the off-diagonal mass of the kernel at that power. This yields 900 comparisons, not 900 independent datasets.

Joint accuracy selection uses ranks 32–512, a cut tolerance of 0.05 and probe counts up to 32,768. For R rank stages and Q pair-count stages, selection checks allocate failure probability $0.025/(RQ)$ each. An independent final audit uses failure probability 0.025. A union bound gives at least 95% coverage for selection and final auditing together under the bounded-sampling assumptions. Fixed-size probe grids use the same policy with equal initial and maximum pair counts. Diagnostics record pair counts, allocation decisions and any failure to establish the requested tolerance. Model test outcomes are not used to select rank or audit budgets.

CPU comparisons pin workers to disjoint pairs of cores, limit native thread pools to two threads and disable CUDA visibility. Wall-clock split times include factorization, assignment, refinement and any method-specific graph or clustering work; feature loading, imports and quality scoring are excluded. A small common low-rank warm-up is reported separately. Native memory profiles record fresh-process peak RSS before scoring or matched controls. They include input arrays and framework overhead.

Scaling uses nested samples of packed 2048-bit ChEMBL 37 fingerprints [ChEMBL37], four restarts, two refined candidates, rank 256, and exact or 1% capacity corridors. The plain and combined settings are measured separately. Independent large-data audits use 262,144 uniform pair draws and up to 128 uniformly sampled test queries, each evaluated against the complete training fold. A conservative bounded-mean interval describes query-sampling uncertainty. End-to-end budgets include loading and auditing. Failures to complete within these budgets are reported separately from split-call timings.

5.4 Independent-cohort experiment and uncertainty

A synthetic regression generator supplies fifty observed families with forty entities each. Responses combine a linear function of four stable features, an independent family effect and observation noise. Nonnegative cosine features encode family identity and a weaker continuous signal. Two fixed models represent different inductive assumptions: five-neighbour regression on the full representation and ridge regression supplied with the generative stable features. Ridge regression therefore serves as a privileged invariant reference. Plain and combined splitting use rank 128; the plain cosine kernel uses its exact finite feature map when available.

For twenty prespecified generative seeds, each split retains 1600 training, 200 validation and 200 test entities. Models are fitted on training labels only. We compare held-out RMSE and model rankings with those obtained on two independent cohorts: new observations from families present in the training set, and observations from fifty new families. The family-reference split uses the generator's known family identities and is a mechanism control. Cohort outcomes never enter split construction. This experiment contains no real experiment

dates or prospective application evidence.

Summaries average seeds within each dataset before averaging datasets. DataSAIL configuration comparisons likewise average configurations within datasets first. Descriptive bootstrap intervals resample dataset means or independent synthetic instances, using 10,000 draws and seed 31415. Seeds, models and perturbations of the same dataset are not treated as independent projects. All method settings, realized counts, native outcomes and per-case timing and memory scopes are retained.

5.5 Direct-kernel controls and component attribution

The added controls use all six full molecular datasets and three seeds. A direct-kernel oracle stores the float32 similarity matrix and supplies row-to-fold mass, similarity columns, diagonal terms and cut evaluations to the unchanged `balanced_lloyd` and `fm_polish` routines. The factor arm supplies the same operations through compact coordinates. The two arms start from identical seeded feasible assignments, permit at most 25 assignment iterations and n refinement operations, and use the same acceptance tests. We compare ordinary and cubed Tanimoto similarities at exact counts and a 1% corridor, using 64, 256 and 512 landmarks. The dense arm is a heuristic reference, not an optimal solution. Comparisons of a dense $BB^\top$ oracle with its factor verify that the interface invokes equivalent optimization operations.

Separate 1- and 5-second controls include representation construction and use the same ordered restart seeds. A supervising process stops the timed optimizer at its deadline. Only checkpoints whose objective evaluation and label-array serialization completed before the cutoff are eligible; the initial feasible assignment is retained if construction exhausts the budget. Both arms repeatedly call the current one-step assignment and refinement routines. Scheduling and termination overshoot is recorded, and late work cannot improve the selected partition. The supplement also retains an earlier cooperative protocol that checks time between operations and allows the last operation to finish.

A separate 36-case grid measures dense-versus-factor cost on nested samples of 4,000, 16,000 and 32,000 ChEMBL fingerprints, using three seeds, ranks 64/256/512, ordinary similarity and a 1% corridor. The fixed sampling order matches the existing ChEMBL scaling protocol. Initialization, search limits, two-core allocations, timing scope and independent full-kernel evaluation match the iteration-controlled molecular study. These controls do not replace the existing public-API scaling measurements with their different restart and hardware settings.

Component controls retain the same feasible initialization when assignment or refinement is omitted. Other rows vary one versus four restarts, uniform versus similarity-spreading landmarks, and ordinary versus cubed kernels. The four-start setting refines two candidates. Three additional rows share one capped group array, a factor-independent feasible group packing, and strict preservation of all groups. The grouped optimizer receives that packing as a warm start and one additional seeded restart, and refines one candidate. Grouping changes the feasible set, so its effect is distinguished from optimization within fixed groups. Component timings disclose shared representation-construction costs.

Independent full NumPy float64 Tanimoto scoring evaluates every returned partition. We report absolute cut error, error divided by total off-diagonal kernel mass, and error divided by

the true cut. Both denominators use the same kernel power as the factor. Cut-relative error is undefined at a zero true cut; it is not stabilized by adding an arbitrary constant. These representation errors are distinct from the original-kernel objectives attained by the two optimizers. The earlier 900 full-kernel comparisons are also reanalysed with this cut-relative denominator. Independent RDKit bit-vector calculations replay the added original-kernel scores and check saved counts, groups and CCPart threshold edges.

5.6 CCPart comparison and native constraints

We use the unmodified CCPart implementation from Hestia-GOOD 1.2.2, upstream commit `54b83e2`, with similarity thresholds 0.4, 0.6 and 0.8, requested 80/10/10 sizes and no response-label stratification. Every unordered Tanimoto pair is considered when constructing the threshold graph; no top-k truncation is used. Native timing includes similarity construction, connected components and packing. The three repetitions permute input order and invert that permutation for evaluation. CCPart is deterministic: these repetitions measure order sensitivity rather than stochastic optimizer variability.

We retain native counts, groups and all entities. Empty folds are reported explicitly. When all folds are nonempty, two low-rank interventions use the native assignment as a feasible initialization: one preserves only its exact counts; the other also preserves every connected component. The latter uses the same grouped warm-start budget as the component controls. Their timings include factor construction and optimization but reuse the native groups; they are optimization interventions rather than independently timed group construction pipelines. The count-only intervention can split components; the group-preserving intervention retains CCPart's threshold separation while optimizing aggregate similarity. The full native size inventory accompanies the quality and cost summaries.

5.7 Implementation and reproducibility

The direct-kernel, component and CCPart controls added in this revision use the same frozen numerical package as the primary studies. Their own manifest pins the control adapters, upstream CCPart source, inputs and assignments. The original study records are retained without relabelling their producer versions. The added comparison and independent replay are described in the Supplementary Information.

The primary molecular studies use numerical package v0.2.0.dev3. The ODAC25 case uses the frozen v0.1 factor implementation with a separate exact-count group assignment, rather than the primary v0.2.0.dev3 point-level solver. Its weighted adapter is detailed in the ODAC25 section of the Supplementary Information. AI assistance supported code development, analysis scripting and manuscript preparation. Numerical claims are checked against saved observations, source and assignment hashes, independent kernel calculations and targeted regression tests. The ODAC25 case also includes a separate offline replay and numerical audit; its tables are generated from the verified records.

Earlier-release molecular, materials and assay studies are identified separately in the Supplementary Information. They provide supporting evidence rather than measurements of the primary implementation. In particular, earlier v0.1 H100 timings are retained as archival measurements. Independent NumPy calculations check the saved global and local scores.

Threshold ramps and alternative fingerprints are supporting diagnostics with definitions separate from the primary local score. The software also supports fold-pair weights; the primary benchmarks use the all-fold objective.

Data and code availability

Code, manuscript sources, compiled PDFs, study manifests and saved assignments for the completed revision are available on GitHub main, including the direct-kernel, CCPart and component controls in `revision_20261103_controls`. Earlier primary evidence is preserved in the versioned PALM manuscript release. The primary numerical implementation is pinned to commit `f7e0866`; the delivered source and environment records identify later documentation and manuscript changes and reproduction adapters. Public-input recipes and checksums accompany the records. The earlier v0.1 review release preserves the archival experiments. MoleculeNet, ChEMBL, QMOF, Materials Project and SIMPD inputs retain their original attribution and terms. The ODAC25 addition, frozen adapter, compact evidence and assignment maps are included in `odac25_case` on the ODAC25 revision branch; its dataset sources retain their CC-BY-4.0 attribution. The expanded metadata caches can be regenerated from the pinned column-range requests and are not needed to rebuild the article.

Author contributions

Pending author-supplied CRediT statement.

Funding

Pending author-supplied funding statement.

Competing interests

Pending author-supplied competing-interest statement.

Acknowledgements and use of AI tools

OpenAI Codex and Anthropic Claude assisted with code development and review, analysis and manuscript preparation. The authors are responsible for the scientific claims, references and final submitted manuscript.

References

Zhenqin Wu, Bharath Ramsundar, Evan N. Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S. Pappu, Karl Leswing, and Vijay Pande. MoleculeNet: a benchmark for molecular machine learning. *Chemical Science*, 9(2):513–530, 2018. doi: 10.1039/C7SC02664A.

Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, Andrew Palmer, Volker Settels, Tommi Jaakkola, Klavs Jensen, and Regina Barzilay. Analyzing Learned Molecular Representations for Property Prediction. *Journal of Chemical Information and Modeling*, 59(8):3370–3388, 2019. doi: 10.1021/acs.jcim.9b00237.

Simon Steshin. Lo-Hi: Practical ML Drug Discovery Benchmark. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/cb82f1f97ad0ca1d92df852a44a3bd73-Abstract-Datasets_and_Benchmarks.html.

Robert P. Sheridan. Time-Split Cross-Validation as a Method for Estimating the Goodness of Prospective Prediction. *Journal of Chemical Information and Modeling*, 53(4):783–790, 2013. doi: 10.1021/ci400084k.

Gregory A. Landrum, Maximilian Beckers, Jessica Lanini, Nadine Schneider, Nikolaus Stiefl, and Sereina Riniker. SIMPD: an algorithm for generating simulated time splits for validating machine learning approaches. *Journal of Cheminformatics*, 15:119, 2023. doi: 10.1186/s13321-023-00787-9.

Roman Joeres, David B. Blumenthal, and Olga V. Kalinina. Data splitting to avoid information leakage with DataSAIL. *Nature Communications*, 16:3337, 2025. doi: 10.1038/s41467-025-58606-8.

Roman Joeres, David B. Blumenthal, and Olga V. Kalinina. Addendum: Data splitting against information leakage with DataSAIL. *Nature Communications*, 17:1597, 2026. doi: 10.1038/s41467-025-67495-w.

Christopher K. I. Williams and Matthias Seeger. Using the Nyström Method to Speed Up Kernel Machines. In *Advances in Neural Information Processing Systems*, volume 13, 2000. URL https://proceedings.neurips.cc/paper/2000/hash/19de10adbaa1b2ee13f77f679fa1483a-Abstract.html.

Inderjit S. Dhillon, Yuqiang Guan, and Brian Kulis. Kernel k-means, Spectral Clustering and Normalized Cuts. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 551–556, 2004. doi: 10.1145/1014052.1014118.

Alex Gittens and Michael W. Mahoney. Revisiting the Nyström Method for Improved Large-scale Machine Learning. *Journal of Machine Learning Research*, 17(117):1–65, 2016. URL https://jmlr.org/papers/v17/gittens16a.html.

Shusen Wang, Alex Gittens, and Michael W. Mahoney. Scalable Kernel K-Means Clustering with Nyström Approximation: Relative-Error Bounds. *Journal of Machine Learning Research*, 20(12):1–49, 2019. URL https://jmlr.org/papers/v20/17-517.html.

Radha Chitta, Rong Jin, Timothy C. Havens, and Anil K. Jain. Approximate kernel k-means. In *Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 895–903, 2011. doi: 10.1145/2020408.2020558.

Raúl Fernández Díaz, Lam Thanh Hoang, Vanessa Lopez, and Denis Shields. Effect of dataset partitioning strategies for evaluating out-of-distribution generalisation for predictive models in biochemistry. ISMB 2024 poster; Hestia-GOOD software, 2024. URL https://github.com/IBM/Hestia-GOOD. CCPart implementation: Hestia-GOOD 1.2.2, commit 54b83e2; full source identity accompanies the benchmark.

C.M. Fiduccia and R.M. Mattheyses. A Linear-Time Heuristic for Improving Network Partitions. In *19th Design Automation Conference*, pages 175–181, 1982. doi: 10.1109/dac.1982.1585498.

B. W. Kernighan and S. Lin. An Efficient Heuristic Procedure for Partitioning Graphs. *Bell System Technical Journal*, 49(2):291–307, 1970. doi: 10.1002/j.1538-7305.1970.tb01770.x.

Anuroop Sriram, Logan M. Brabson, Xiaohan Yu, Sihoon Choi, Kareem Abdelmaqsoud, Elias Moubarak, Pim de Haan, Sindy Löwe, Johann Brehmer, Muhammed Shuaibi, John R. Kitchin, Max Welling, C. Lawrence Zitnick, Zachary Ulissi, Andrew J. Medford, and David S. Sholl. An open dataset of gas adsorption in deformable metal–organic frameworks for direct air capture. *Nature Chemical Engineering*, 2026. doi: 10.1038/s44286-026-00445-5.

David Rogers and Mathew Hahn. Extended-Connectivity Fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. doi: 10.1021/ci100050t.

Liva Ralaivola, Sanjay J. Swamidass, Hiroto Saigo, and Pierre Baldi. Graph kernels for chemical informatics. *Neural Networks*, 18(8):1093–1110, 2005. doi: 10.1016/j.neunet.2005.07.009.

Lars Gottesbüren, Tobias Heuer, Nikolai Maas, Peter Sanders, and Sebastian Schlag. Scalable High-Quality Hypergraph Partitioning. *ACM Transactions on Algorithms*, 20(1):1–54, 2024. doi: 10.1145/3626527.

Saiveth Hernández-Hernández and Pedro J. Ballester. On the Best Way to Cluster NCI-60 Molecules. *Biomolecules*, 13:498, 2023. doi: 10.3390/biom13030498.

Qianrong Guo, Saiveth Hernandez-Hernandez, and Pedro J. Ballester. UMAP-based clustering split for rigorous evaluation of AI models for virtual screening on cancer cell lines. *Journal of Cheminformatics*, 17(1):94, 2025. doi: 10.1186/s13321-025-01039-8.

Guy W. Bemis and Mark A. Murcko. The Properties of Known Drugs. 1. Molecular Frameworks. *Journal of Medicinal Chemistry*, 39(15):2887–2893, 1996. doi: 10.1021/jm9602928.

Darko Butina. Unsupervised Data Base Clustering Based on Daylight's Fingerprint and Tanimoto Similarity: A Fast and Automated Way To Cluster Small and Large Data Sets. *Journal of Chemical Information and Computer Sciences*, 39(4):747–750, 1999. doi: 10.1021/ci9803381.

ChEMBL37. ChEMBL database release 37, 2026. URL https://doi.org/10.6019/CHEMBL.database.37.